1 Comparison Between the Original and Modified FreeCAD Python Code

The original output generated by the fine-tuned CAD-Coder model contains the beginning of a valid FreeCAD modeling sequence, but the generation is incomplete. The model was originally designed to generate CAD-query code rather than a complete, executable FreeCAD Python script. As a result, the generated output reaches the token limit before completing the intended modeling operation.

The modified version addresses these limitations by adding the required Python and FreeCAD imports, initializing the FreeCAD document, reconstructing the missing sketch geometry, and completing the final Pocket operation.

1.1 Added FreeCAD and Python Module Imports

The modified code begins by explicitly importing the modules required by the generated commands:

```
import FreeCAD
import FreeCAD as App
import Part
import PartDesign
import Sketcher
import Mesh
import ProfileLib.RegularPolygon

App.newDocument()
```

Listing 1: FreeCAD and Python module imports added to the generated script

These imports were necessary because the original model output assumes that the FreeCAD environment and the relevant modules are already available. When the generated code is executed as a standalone Python script, these dependencies need to be explicitly imported.

In particular:

- `FreeCAD` and `FreeCAD as App` provide access to the FreeCAD application and document API.
- `Part` is required for creating geometric entities such as circles.
- `PartDesign` provides Part Design features such as Pads and Pockets.
- `Sketcher` is required for creating and applying sketch constraints.
- `Mesh` provides FreeCAD mesh functionality.
- `ProfileLib.RegularPolygon` is required for generating the regular polygon used in the first sketch.

Therefore, these additions make the generated output more self-contained and suitable for execution outside an already initialized FreeCAD environment.

1.2 Added Document Initialization

The line

```
App.newDocument()
```

was added to explicitly create a FreeCAD document before accessing the object named `Unnamed`.

The original code immediately uses commands such as:

```
App.getDocument('Unnamed')
```

without showing where the document was created. Although this may work when the code is executed inside an already initialized FreeCAD session, it is not reliable for a standalone script.

Adding document initialization ensures that a FreeCAD document exists before the generated objects are created.

1.3 Preservation of the Original Sketch and Pad Generation

The first part of the original output was retained because it represents a valid sequence for creating the initial model.

The generated sequence performs the following operations:

1. Creates a Part Design Body.
2. Creates a sketch on the XY plane.
3. Generates a regular hexagonal profile.
4. Applies geometric constraints to the sketch.
5. Creates a Pad from the sketch.
6. Sets the pad length and other parameters.
7. Creates a second sketch.

This portion of the generated output was therefore not fundamentally modified. The primary problem occurs after the creation of the second sketch, where the model stops generating valid code.

1.4 Reconstruction of the Missing Geometry in `Sketch001`

The most important modification was the addition of the following block:

```
ActiveSketch = App.getDocument('Unnamed').getObject('
    Sketch001')

lastGeoId = len(ActiveSketch.Geometry)

geoList = []
geoList.append(
    Part.Circle(
        App.Vector(0.000000, 0.000000, 0.000000),
        App.Vector(0.000000, 0.000000, 1.000000),
        15.000000
    )
)

App.getDocument('Unnamed').getObject(
    'Sketch001'
).addGeometry(geoList, False)

del geoList

constraintList = []

App.getDocument('Unnamed').getObject(
    'Sketch001'
).addConstraint(
    Sketcher.Constraint('Diameter', 0, 16)
)

App.getDocument('Unnamed').getObject(
    'Sketch001'
).addConstraint(
    Sketcher.Constraint('Coincident', 0, 3, -1, 1)
)

App.ActiveDocument.recompute()
App.getDocument('Unnamed').recompute()
```

Listing 2: Reconstructed geometry and constraints for Sketch001

This block was added because the original model successfully created `Sketch001`, but stopped before defining its geometry.

Creating an empty sketch is not sufficient to produce the intended CAD feature. Therefore, the missing geometry had to be reconstructed manually.

A circular geometry was added to `Sketch001`, followed by constraints defining its size and position. This reconstructs the missing portion of the modeling sequence that the CAD-Coder failed to generate because of the output token limitation.

1.5 Change from Pad to Pocket

The original output begins another Pad operation:

```
App.getDocument('Unnamed').getObject('Body').newObject(
    'PartDesign::Pad','Pad001'
)
```

However, the intended operation for the second sketch is subtractive rather than additive. Therefore, the operation was changed from a `Pad` to a `Pocket`:

```
App.getDocument('Unnamed').getObject('Body').newObject(
    'PartDesign::Pocket','Pocket'
)
```

This is an important semantic correction.

A `Pad` adds material to an existing solid, whereas a `Pocket` removes material using the geometry defined by a sketch. Since `Sketch001` contains a circular profile intended to create a hole or cut in the previously created solid, a Pocket is the appropriate Part Design operation.

1.6 Correction of the Profile Reference

The incomplete generated code attempted to use the original sketch as the profile for the new feature:

```
App.getDocument('Unnamed').getObject('Pad001').Profile = (
    App.getDocument('Unnamed').getObject('Sketch'), ['',]
)
```

This was changed so that the Pocket references `Sketch001`:

```
App.getDocument('Unnamed').getObject('Pocket').Profile = (
    App.getDocument('Unnamed').getObject('Sketch001'), ['',]
)
```

This correction is necessary because the newly created circular geometry belongs to `Sketch001`, rather than the original `Sketch`.

The Pocket must therefore reference `Sketch001` so that the circular geometry is used as the profile for the subtractive operation.

1.7 Completion of the Pocket Parameters

The generated output was also completed with the required Pocket parameters:

```
App.getDocument('Unnamed').getObject('Pocket').Direction =
    (0, 0, 1)

App.getDocument('Unnamed').getObject(
    'Pocket'
).AlongSketchNormal = 1
```

```
6
7  App.getDocument('Unnamed').getObject(
8      'Pocket'
9  ).SideType = 2
10
11 App.getDocument('Unnamed').recompute()
12
13 App.getDocument('Unnamed').getObject(
14     'Sketch001'
15 ).Visibility = False
```

Listing 3: Additional Pocket parameters added after the truncated output

These parameters define the direction and behavior of the subtractive extrusion.

The final recompute operation updates the FreeCAD model, while setting the visibility of `Sketch001` to false hides the construction sketch after the Pocket has been created.

1.8 Removal of the Corrupted Generated Code

The original output terminates with a clearly corrupted and incomplete sequence:

```
App.getDocument('Unnamed').getObject(
    'Sketcher.getObject('Sketcher('Sketcher('Sketcher('
    Sketcher(...
```

This is not a valid FreeCAD Python command. It appears to be a consequence of the model reaching its token-generation limit while attempting to continue generating the command sequence.

The invalid fragment was therefore commented out rather than executed:

```
# App.getDocument('Unnamed').getObject(
#     'Sketcher.getObject('Sketcher('Sketcher('Sketcher(...
```

Commenting out this fragment prevents it from causing a Python syntax error while preserving the original generated output for reference.

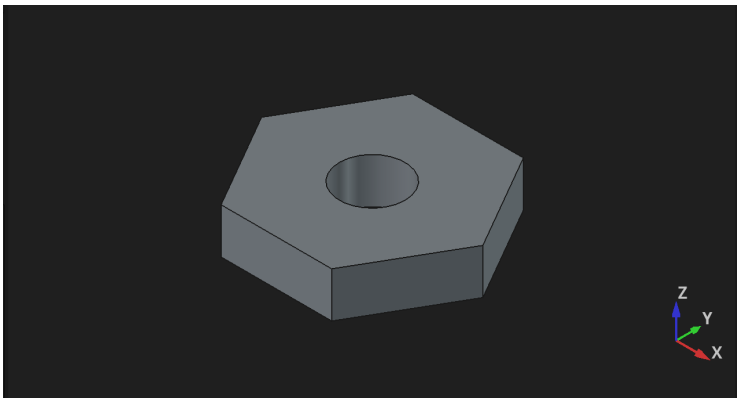


Figure 1: image of output after editing

1.9 Overall Comparison

Table 1 summarizes the main differences between the original and modified outputs.

Table 1: Comparison between the original CAD-Coder output and the modified FreeCAD Python script.

Aspect	Original CAD-Coder Output	Modified Output
Python imports	Missing	Required FreeCAD and Python modules were explicitly imported.
FreeCAD document initialization	Assumed to already exist	An explicit FreeCAD document is created using `App.newDocument()`.
Initial hexagonal sketch	Generated	Preserved.
Initial Pad	Generated	Preserved.
Second sketch	Created but incomplete	Geometry and constraints were added.
Circular profile	Missing	A circular geometry was added to `Sketch001`.
Circular constraints	Missing	Diameter and coincident constraints were added.
Final operation	Incomplete Pad generation	Corrected to a Pocket operation.
Pocket profile	Not completed	Correctly linked to `Sketch001`.
Pocket parameters	Incomplete	Direction, normal direction, side type, and visibility were completed.
Corrupted final output	Present	Invalid generated code was commented out.
Standalone execution	Not reliably executable	More self-contained and executable as a FreeCAD Python script.

1.10 Summary of the Modifications

The modifications can be grouped into four main categories:

1. **Execution setup:** Required modules were imported and a FreeCAD document was initialized.

2. **Geometry reconstruction:** The missing circular geometry and its constraints were added to `Sketch001`.

3. **Feature correction:** The incomplete Pad operation was replaced with a Pocket, and the feature profile was corrected to reference `Sketch001`.

4. **Output repair:** The corrupted portion generated at the end of the token-limited output was commented out, and the missing Pocket parameters were added.

1.11 Conclusion

The modifications were primarily required because the fine-tuned CAD-Coder model generated only a partial FreeCAD script due to the output token limitation. The original output successfully captured the beginning of the modeling procedure but stopped while generating the second feature.

The missing portion was therefore reconstructed by adding the required sketch geometry, constraints, and final Part Design operation. The modified script additionally makes the generated output more suitable for standalone execution by explicitly importing the required FreeCAD modules and initializing the document.

The final Pad operation was changed to a Pocket and its profile was corrected to reference `Sketch001`, ensuring that the generated CAD sequence corresponds to the intended subtractive modeling operation.

Overall, the modifications do not replace the model-generated design. Instead, they **complete and repair the truncated output produced by the fine-tuned CAD-Coder model**, allowing the generated CAD instructions to be executed as a more complete FreeCAD Python script.